

Práctica 2-DSA

Alumnos:

- Alvarez Jeremías Martín
- Palacios Tomás Agustin

Ejercicio 1 - Broken Access Control

Reto **ejercicio1** (<https://tp2-ejercicio1.dsa.linti.unlp.edu.ar/>)

Loguearse con:

Usuario: *user*
Password: *pass*

Objetivo: Obtener las tarjetas de crédito de otros usuarios. En el CTF la flag estará en la tarjeta de crédito del usuario “admin”

A primera vista, podemos ver que una vez que ingresamos los datos provistos por la cátedra para el login como vemos a continuación:

Login

Username:

Password:

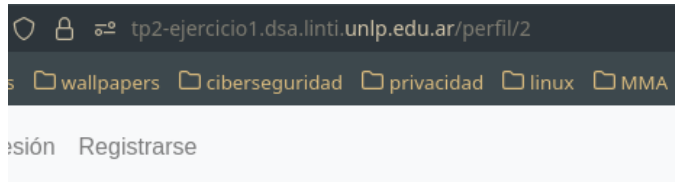
Login

Podemos encontrar un dato inicial muy valioso, en la URL no sólo vemos que nos encontramos viendo nuestro perfil, sino que además seguido se muestra un número que podría indicar en el mejor de los casos que se trata del identificador que tiene cada usuario.



Lo siguiente que podemos hacer es modificar ese valor, esperando que no nos sea denegado el acceso y podamos ver la información de otros usuarios.

Veámoslo:



Perfil de usuario2

Nombre: Nombre2

Apellido: Apellido2

Tarjeta de crédito: 8291-4473-5462-3941

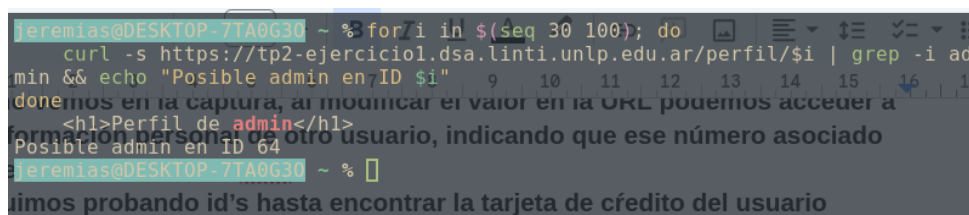
Como vemos en la captura, al modificar el valor en la URL podemos acceder a la información personal de otro usuario, indicando que ese número asociado pertenece al id de cada user.

Seguimos probando id's hasta encontrar la tarjeta de crédito del usuario admin pero nos va a convenir realizar una automatización de peticiones ya que hay por lo menos 20 usuarios existentes. Para ahorrarnos el trabajo de incrementar el valor una y otra vez escribimos un script sencillo que básicamente itera de 1 a 100 (rango elegido a criterio) y chequea si aparece la palabra "admin" (esta búsqueda es una forma sencilla que elegimos, podría haber alguna más eficiente, como por ejemplo buscar por regex en pos de hallar algo que coincida con "flag{...}").

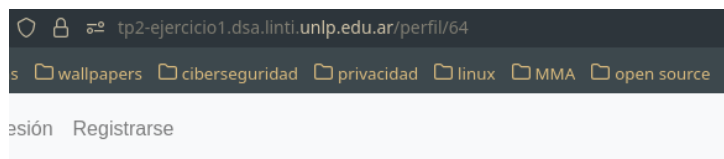
Nuestro algoritmo queda de la siguiente forma:

```
for i in $(seq 30 100); do
curl -s https://tp2-ejercicio1.dsa.linti.unlp.edu.ar/perfil/$i | grep -i admin && echo "Posible admin en ID $i"
done
```

Este script nos devuelve el siguiente resultado:



Como podemos ver, encontramos en /perfil/64 la flag:



Una vez hallada la vulnerabilidad que permite acceder a la información personal de cada usuario mediante la simple tarea de modificar el número que figura en la web (identificador de cada uno), **¿Cómo podemos hacer para solucionarlo?**

Una primera idea que puede surgir sería:

¿Y si ofuscamos la URL para que el atacante a primera vista no pueda darse cuenta de la información valiosa que estamos exponiendo en la URL?

Como vimos en la teoría y lo pudimos confirmar en el caso práctico ocurrido en 2013 con el Padrón Electoral, **OFUSCAR NO ES ENCRIPTAR**.

Es decir, no alcanza con ocultar el ID, usar UUIDs largos, esconder botones o validar solo desde el frontend. Esto puede “complicar” un poco el ataque (por poco tiempo), pero no corrige la vulnerabilidad.

La seguridad sí o sí tiene que estar en el backend.

Una posible solución sería, del lado del servidor, validar de quién proviene cada petición y en base a los permisos que ese usuario tenga, mostrarle o no determinada información (a continuación vamos a expandir esta idea). Es decir: cuando alguien pide “.../perfil/35”, el backend no debería preguntar solamente “¿existe el usuario 35?”, sino también “¿el usuario que está logueado tiene permiso para ver/modificar el usuario 35?”. Si no tiene permiso, debe rechazar la solicitud.

Ahora bien, un usuario por más que tenga permisos, no debería poder ver información sensible de cada usuario ya que esto indicaría un riesgo muy grande en caso de que un atacante pueda por algún medio acceder al usuario con privilegios o usuario administrador. Por lo tanto, en base a las reglas de autorización definidas, cada usuario privilegiado debería poder acceder a la información que le concierne dado su rol, pero jamás debería otorgarle el servidor acceso a datos personales innecesarios.

Ejercicio 2 - Insecure DOR (Order Tickets)

Para los ejercicios 2, 3, 4 y 5 se utilizará la aplicación **bWAPP** ("an extremely buggy web app" <https://sourceforge.net/projects/bwapp/files/bWAPP/>, <http://www.itsecgames.com/index.htm>).

Utilizando el navegador ingresar a <http://localhost:12000/install.php> (por defecto los lleva a /login.php y tira un error porque todavía no está creada la DB y seteada la config de la aplicación).

Clic en el enlace "Click **here** to install".

Login con user/pass: **bee/bug**

Luego ingresar a http://localhost:12000/security_level_set.php y configurar el nivel de seguridad en "**low**".

En el menú "Bugs" (Portal) buscar en la lista los retos de la categoría "A4 - Insecure Direct Object References" y "A7 - Missing Functional Level Access Control".

Reto **bWAPP** (http://localhost:12000/insecure_direct_object_ref_2.php)

Esta página simula un sitio para comprar/encargar entradas de cine por un valor de "15 EUR".

Objetivo a cumplir: encargar tickets gratis o por un valor distinto.

Para este ejercicio, podemos ver un formulario en donde el usuario ingresa la cantidad de tickets que desea y obtiene como respuesta el valor total.

Ahora bien, si inspeccionamos el código fuente (html), podemos encontrar un input oculto (atributo hidden) que corresponde al valor de cada ticket.

```

<h1>
  Insecure DOR (Order Tickets)
</h1>
<form action="/insecure_direct_object_ref_2.php" method="POST">
  <p></p>
  <p>
    I would like to order
    <input type="text" name="ticket_quantity" value="1" size="2">
    tickets.
  </p>
  <input type="hidden" name="ticket_price" value="15">
  <button type="submit" name="action" value="order">Confirm</button>
</form>
<br>
</div>
<div id="side"></div>
<div id="disclaimer">

```

Si encargo 5 tickets, lógicamente obtendré el total acorde a esa cantidad:

/ Insecure DOR (Order Tickets) /

How many movie tickets would you like to order? (15 EUR per ticket)

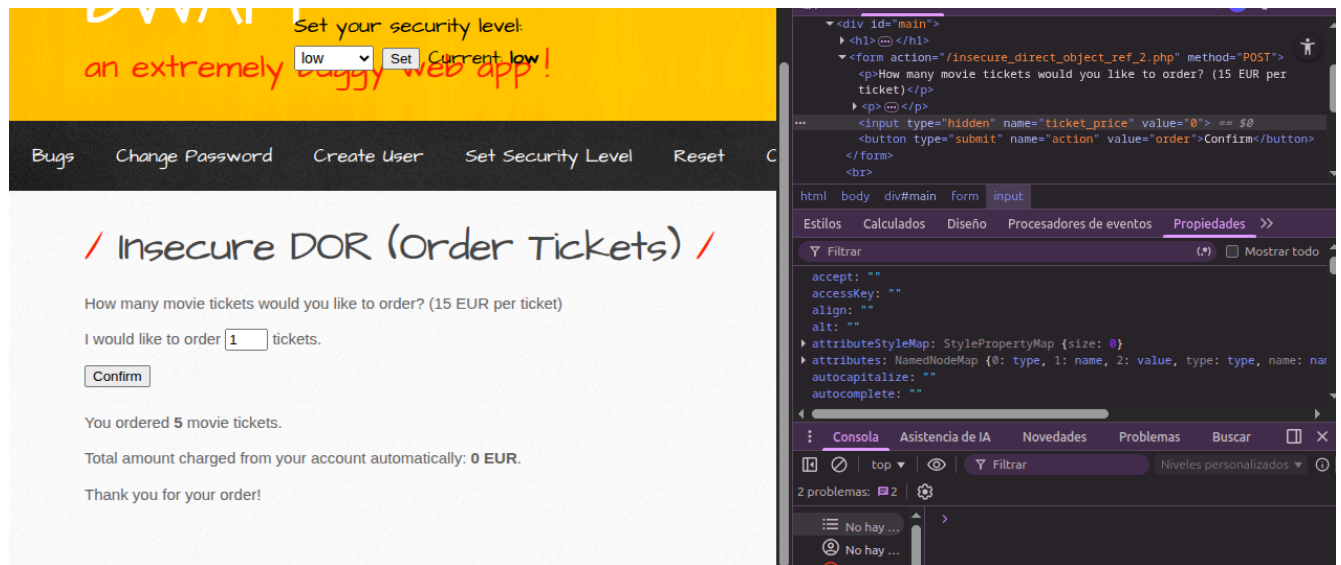
I would like to order tickets.

You ordered **5** movie tickets.

Total amount charged from your account automatically: **75 EUR**.

Thank you for your order!

Para explotar esta vulnerabilidad basta con realizar la modificación manual del valor asociado al input oculto con el valor deseado (en nuestro caso gratis):



Como podemos observar en el lateral derecho, cambiamos el value a 0 y posteriormente enviamos la petición al servidor, el cuál nos devuelve que el valor total a abonar es de 0 EUR.

¿Cómo podemos solucionar esta confianza en el cliente (elijo el precio que quiero y se asume correcto)?

En nuestro caso el servidor está aceptando como verdadero un dato que viene del navegador, cuando el navegador siempre puede ser manipulado.

Por lo tanto, el precio NUNCA debe establecerse en el HTML. Por más que este oculto a los ojos del usuario (atributo hidden) con sólo abrir, por ejemplo, la consola de desarrollador del navegador web podemos encontrarlo y atacarlo.

Cuando llega el pedido al backend, el servidor lo que tiene que hacer es buscar el precio real en su propia base de dato. Eventualmente calculará el total, si se aplica algún descuento, impuesto, verificar stock, etc y recién ahí devolver la orden de compra al usuario.

SIEMPRE calcular estos datos en el backend, si el usuario quiere probar y enviarme valores de precios como servidor sólo que ignorarlos y obtener la información válida de MI base de datos.

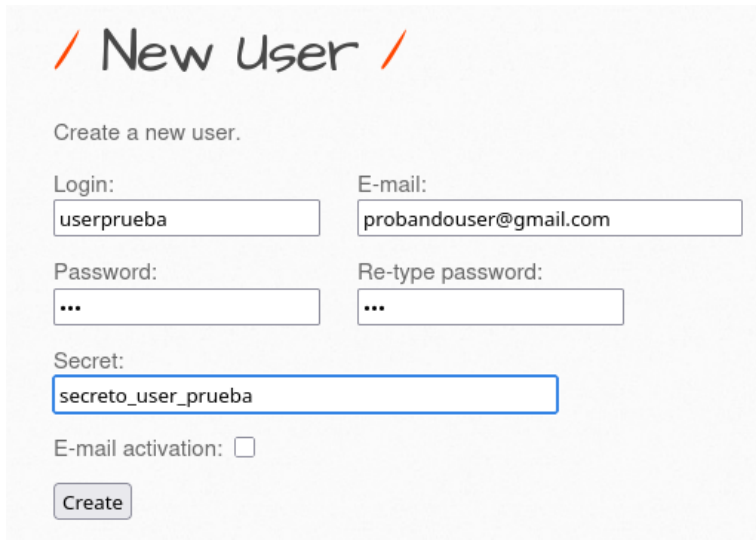
Ejercicio 3 - Insecure DOR (Change Secret)

Reto **bWAPP** (http://localhost:12000/insecure_direct_object_ref_1.php)

Esta página permite cambiar el “Secret” de tu usuario (ver http://localhost:12000/user_extra.php).

Objetivo a cumplir: cambiar el “Secret” de otro usuario que no sea el logueado. Verificar que se cambió correctamente el secret logueando con el usuario víctima, accediendo a http://localhost:12000/insecure_crypt_storage_1.php y viendo el localStorage

Para este ejercicio contamos con el usuario “bee” con pass “bug” y secreto “hola”. Cabe destacar además que registramos un segundo usuario llamado “userprueba” con pass “123” y secreto “secreto_user_prueba”.



New User

Create a new user.

Login:

E-mail:

Password:

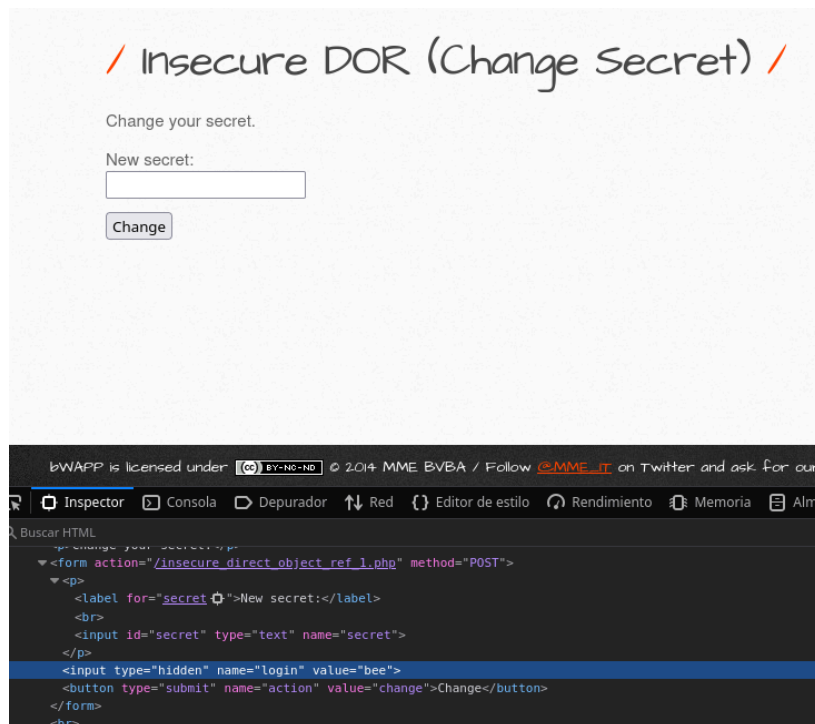
Re-type password:

Secret:

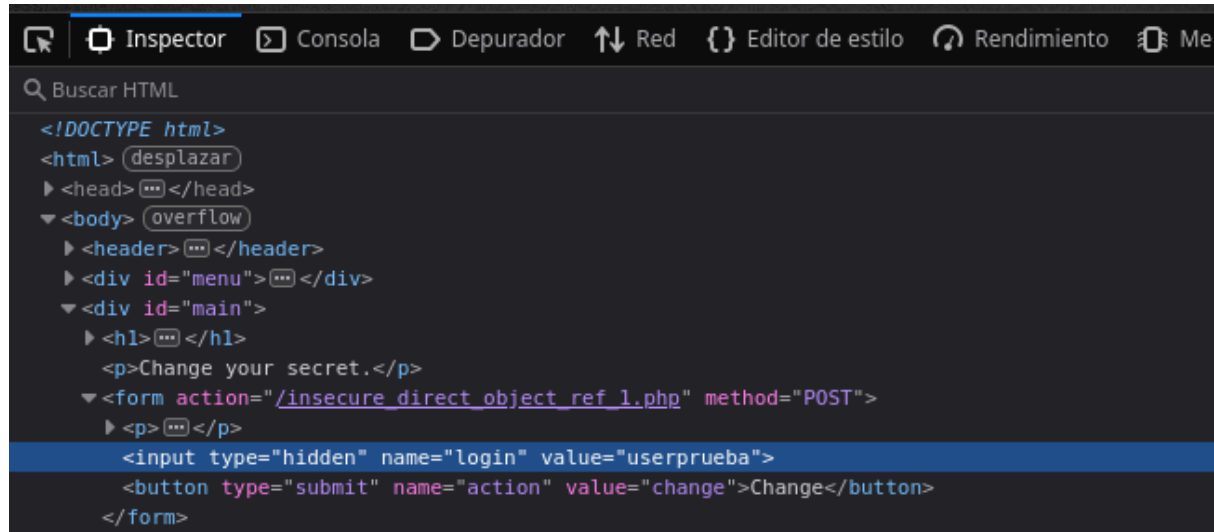
E-mail activation: ☐

Posteriormente nos logueamos con el usuario “bee” y nos redirigimos a la página [“http://localhost:12000/insecure_direct_object_ref_1.php”](http://localhost:12000/insecure_direct_object_ref_1.php) para cambiar nuestro secreto.

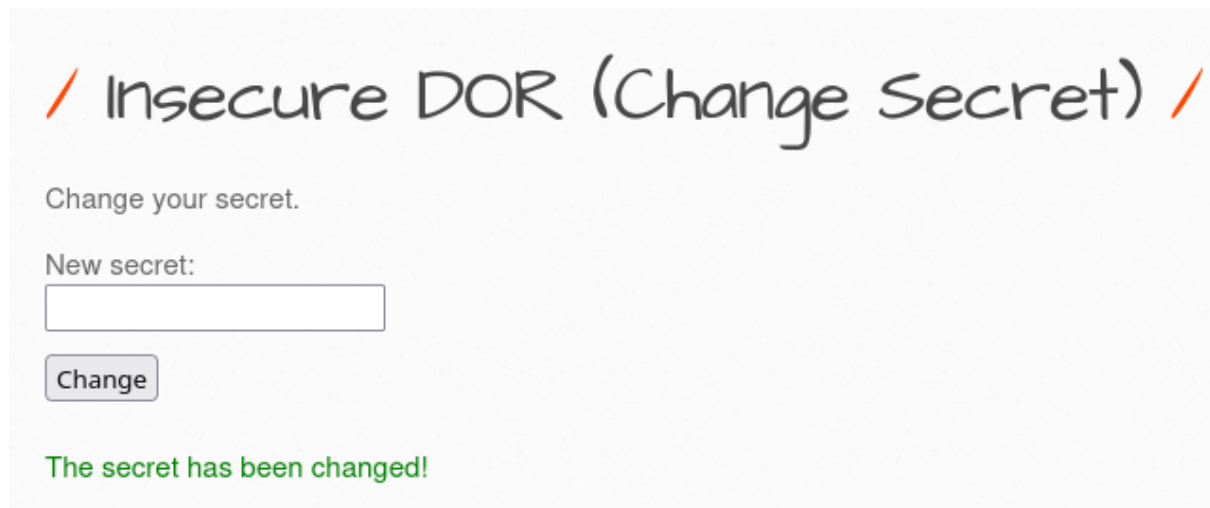
Si observamos el código fuente HTML podemos notar el mismo error que aparecería en el ejercicio anterior, input oculto con valor seteada por defecto (en este caso nombre de usuario).



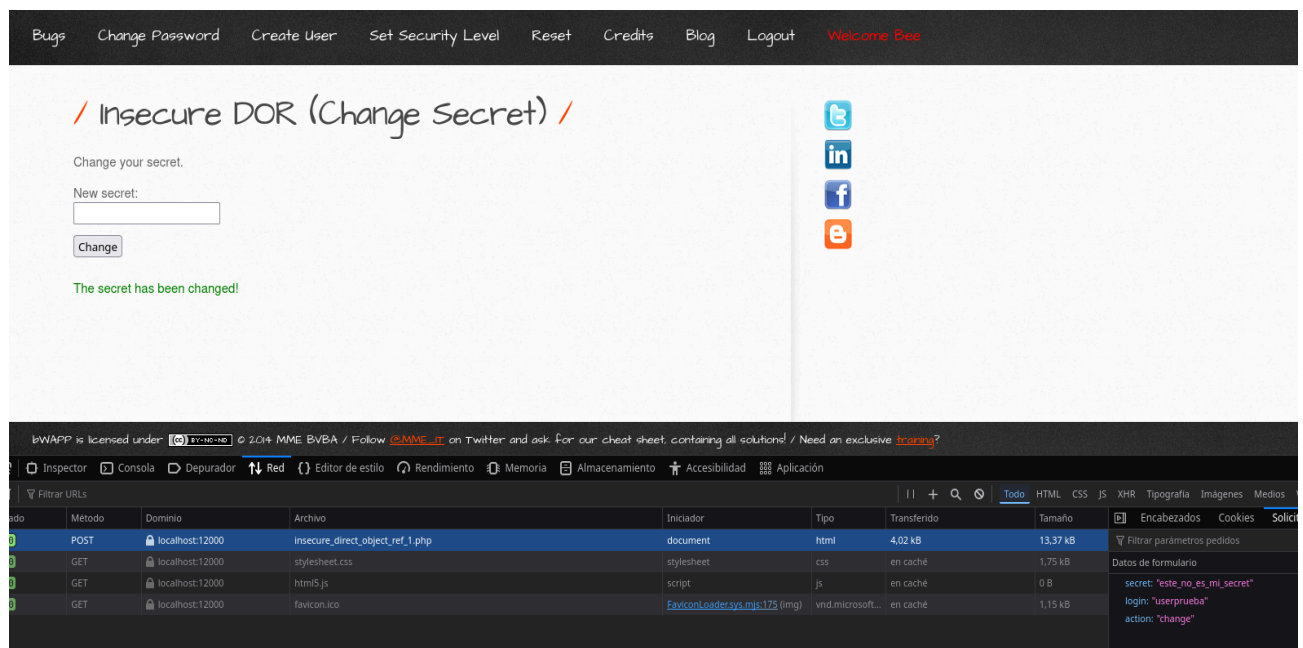
Este dato nos brinda, ya de forma intuitiva, la idea de que si cambio el value por algún nombre de usuario existente podré realizar modificaciones suplantando su identidad. Probemos cambiar el value por “prueba” (segundo usuario que creamos anteriormente):



Ahora completamos el formulario, pongamos “este_no_es_mi_secret” para chequear si puedo cambiar el secreto del usuario ingresado modificando el input hidden.



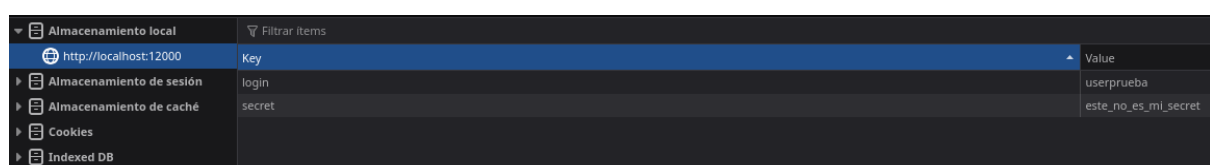
Si vemos el POST del formulario:



Podemos ver en la esquina inferior derecha el secret es “este_no_es_mi_secret” y el usuario asociado no es bee (aunque la web nos muestra “Welcome Bee”) sino que es “userprueba” junto al action “change”.

Esto nos indica que oficialmente modificamos el secreto de otro usuario en lugar del nuestro.

Si revisamos el localStorage podremos ver como quedó:



¿Cómo podemos solucionar el Insecure DOR (Change Secret)?

Al igual que el ejercicio de los tickets, el servidor NUNCA debe aceptar que el cliente decida qué secreto cambiar modificando el input hidden asociado a un usuario determinado.

Si el nombre seteado en el input difiere del nombre de usuario de la sesión, NO debe aceptar lo ingresado por el formulario.

El servidor tiene que encargarse de la modificación desde el backend, sin haber inputs ocultos en el código HTML.

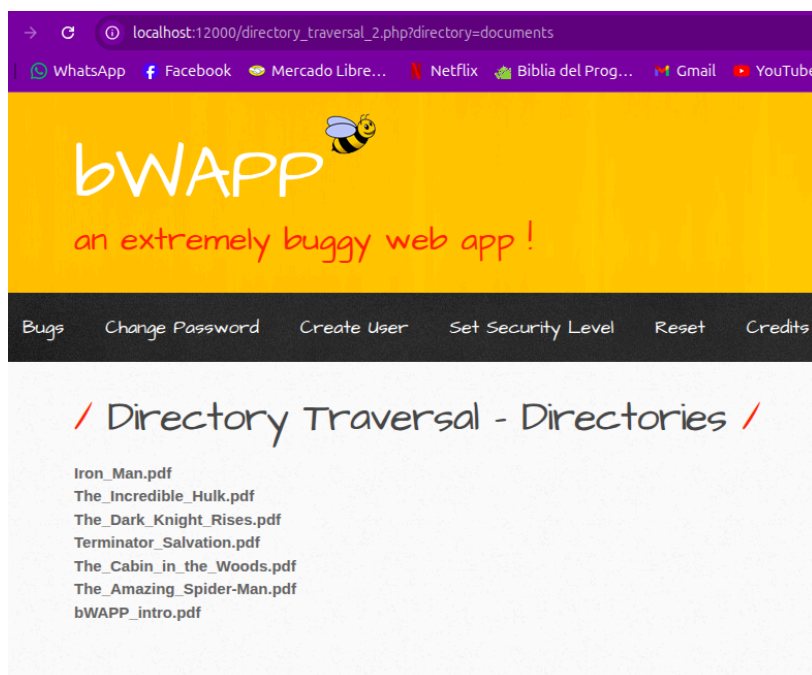
Ejercicio 4 - Directory Traversal

Reto **bWAPP** (http://localhost:12000/directory_traversal_2.php?directory=documents)

Reto **bWAPP** (http://localhost:12000/directory_traversal_1.php?page=message.txt)

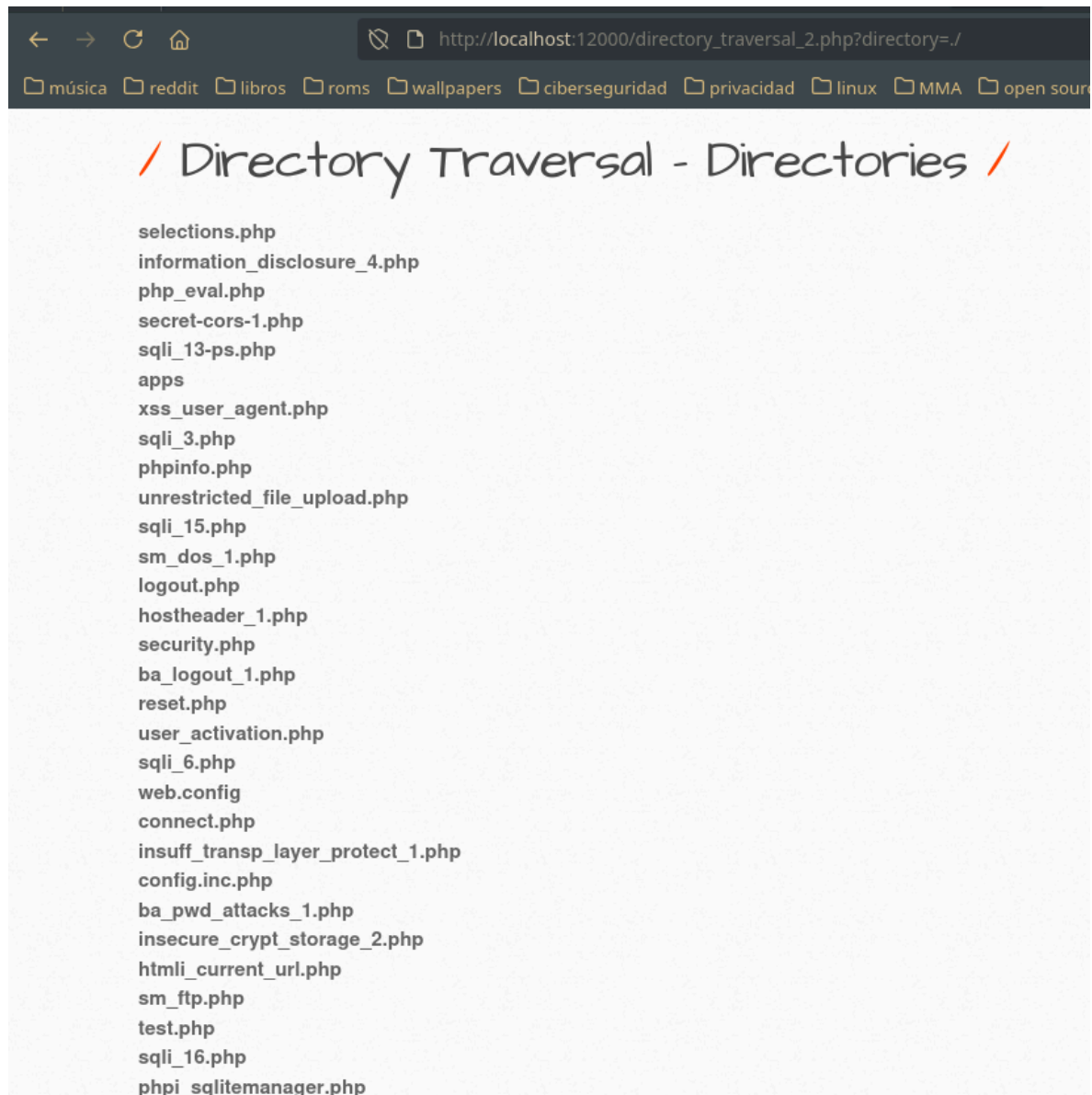
Objetivo a cumplir: listar los usuarios del sistema operativo subyacente (sistema del contenedor docker).

En este ejercicio a simple vista podemos ver una especie de file system, se listan documentos provenientes de un directorio... es decir, mantiene una estructura típica de archivos.



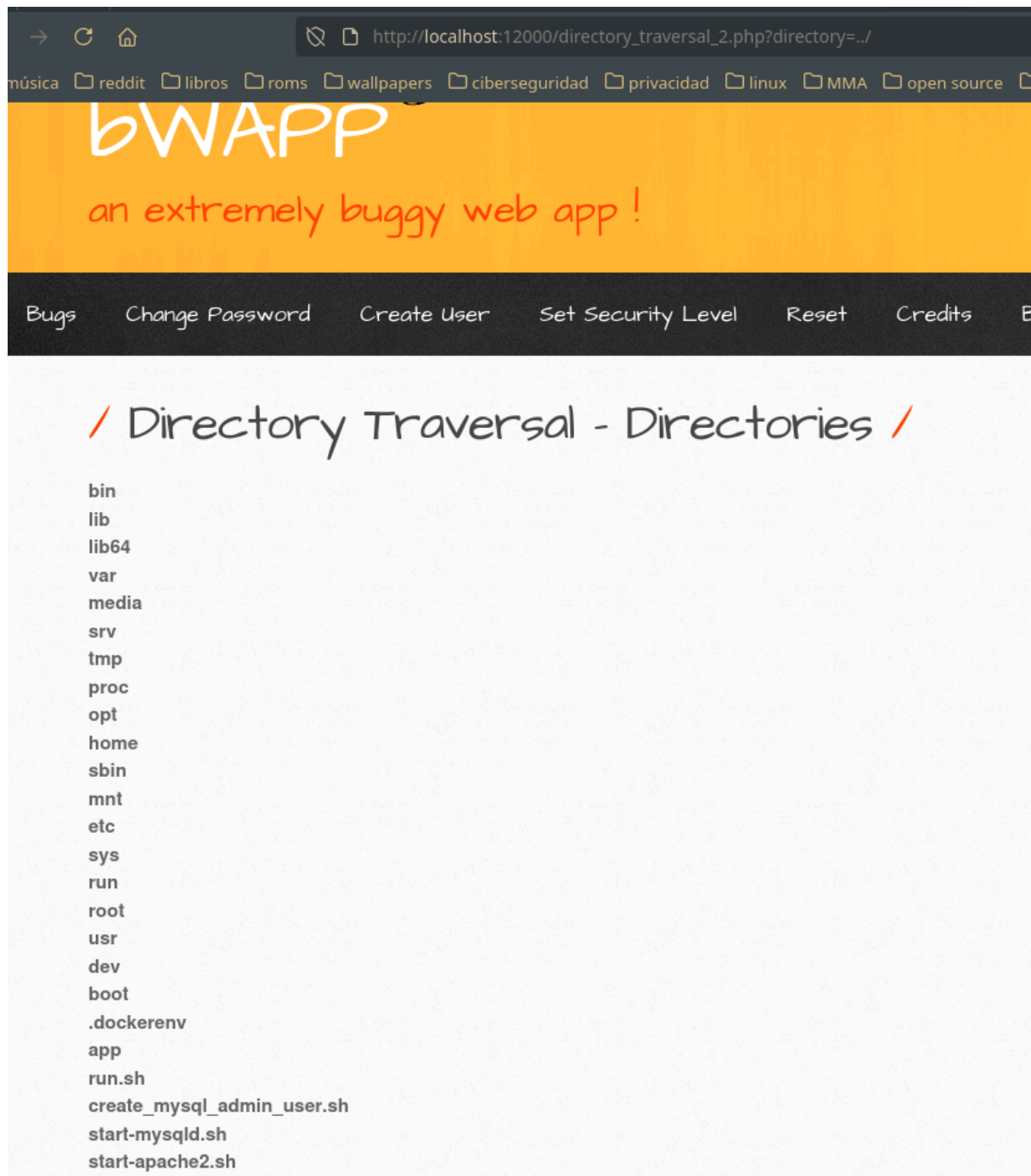
Ahora bien, ¿Qué pasa si reviso un poco más ese file system?

Probemos moviéndonos sobre su estructura, si revisamos que más puede haber en el mismo nivel de “documents”:



Acá ya aparecen cosas copadas, archivos php, carpetas, etc.

¿Y si retrocedemos un nivel en su estructura mediante “../”?



Luego si accedemos a `/etc`:



La aplicación devuelve el listado completo de archivos dentro de /etc.

Entre ellos aparece el archivo “passwd”, junto con otros como “shadow”, “group”, “hosts”, “sudoers”, etc.

Como podemos ver el sistema permite manipular el parámetro directory de la URL para acceder a cualquier carpeta de su file system.

Si entramos en passwd obtenemos información de cada usuario:



¿De qué forma podemos solucionar esta vulnerabilidad?

Como solución, NUNCA se deberían exponer rutas reales del sistema. Si bien el usuario siempre puede “completar” o modificar la URL desde su navegador, lo que podemos hacer nosotros es que el backend no acepte cualquier ruta que el usuario escriba, debemos mostrar sólo lo que nosotros queremos que el usuario vea. Es decir, podemos limitar el “movimiento” del usuario por el file system permitiéndole el acceso a la carpeta actual y SÓLO en los archivos que dejamos seteados como visibles, por ejemplo, tener un txt con:

“
crimen_y_castigo.pdf
Los_Hermanos_Karamazov.pdf
La_muerte_de_Ivan_Illich.pdf
“

El usuario podrá ver en el directorio actual sólo estos archivos y el servidor no acepta el acceso al usuario a ninguna otra path.

Ejercicio 5 - Restrict Device Access

Reto **bWAPP** (http://localhost:12000/restrict_device_access.php)

Esta página solo muestra su contenido cuando se la accede con un dispositivo móvil.

Objetivo a cumplir: acceder sin usar un dispositivo móvil. (Ayuda: pueden utilizar Mozilla Firefox y ver la “respuesta”)

Para este ejercicio se nos aclara de antemano que el contenido sólo podremos verlo accediendo desde un dispositivo móvil.

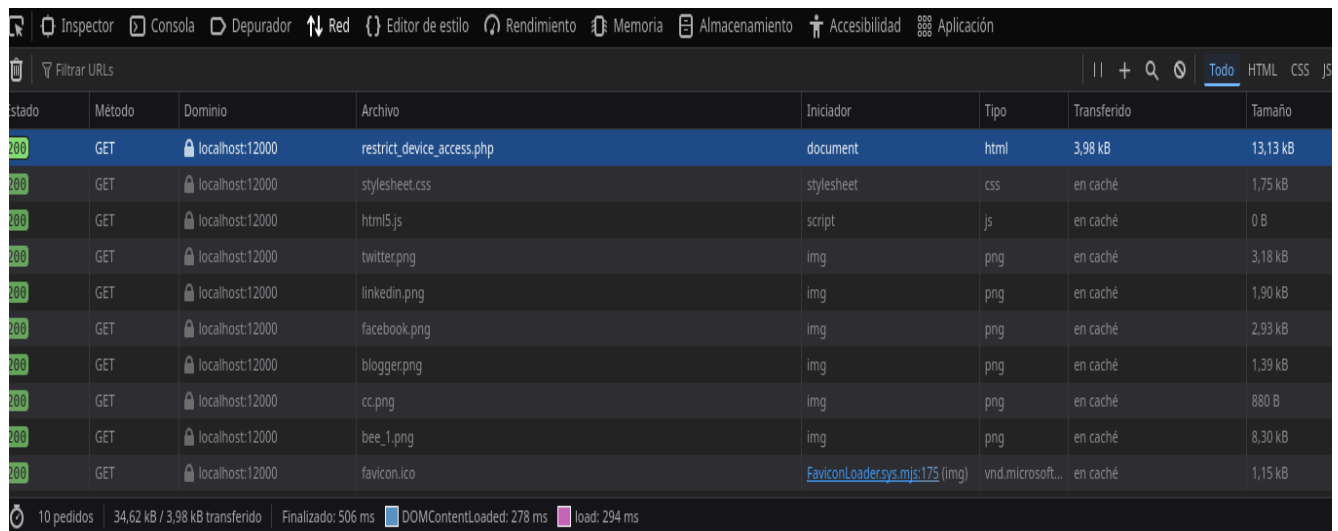
Esto podemos confirmarlo ingresando al sitio en cuestión:



Ahora bien, ¿Podemos acceder a su contenido sin necesidad de conectarnos desde un celular?

Veamos una posible solución utilizando Firefox.

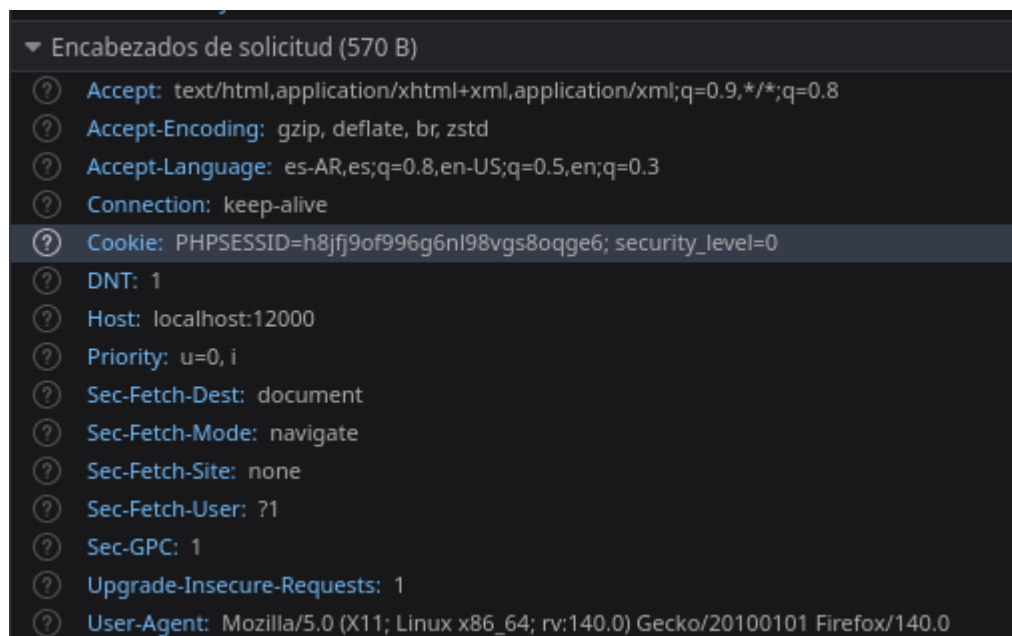
Si abrimos la pestaña Network (Red) del panel del navegador web (f12) podremos ver la petición a “restrict_device_access.php”:



Estado	Método	Dominio	Archivo	Iniciador	Tipo	Transferido	Tamaño
200	GET	localhost:12000	restrict_device_access.php	document	html	3,98 kB	13,13 kB
200	GET	localhost:12000	stylesheet.css	stylesheet	css	en caché	1,75 kB
200	GET	localhost:12000	html5.js	script	js	en caché	0 B
200	GET	localhost:12000	twitter.png	img	png	en caché	3,18 kB
200	GET	localhost:12000	linkedin.png	img	png	en caché	1,90 kB
200	GET	localhost:12000	facebook.png	img	png	en caché	2,93 kB
200	GET	localhost:12000	blogger.png	img	png	en caché	1,39 kB
200	GET	localhost:12000	cc.png	img	png	en caché	880 B
200	GET	localhost:12000	bee_1.png	img	png	en caché	8,30 kB
200	GET	localhost:12000	favicon.ico	FaviconLoadersys.mjs:175 (img)	vnd.microsoft...	en caché	1,15 kB

10 pedidos | 34,62 kB / 3,98 kB transferido | Finalizado: 506 ms | DOMContentLoaded: 278 ms | load: 294 ms

Si ahora revisamos el panel de Encabezados (lateral derecho) podremos ver los siguientes headers de request:

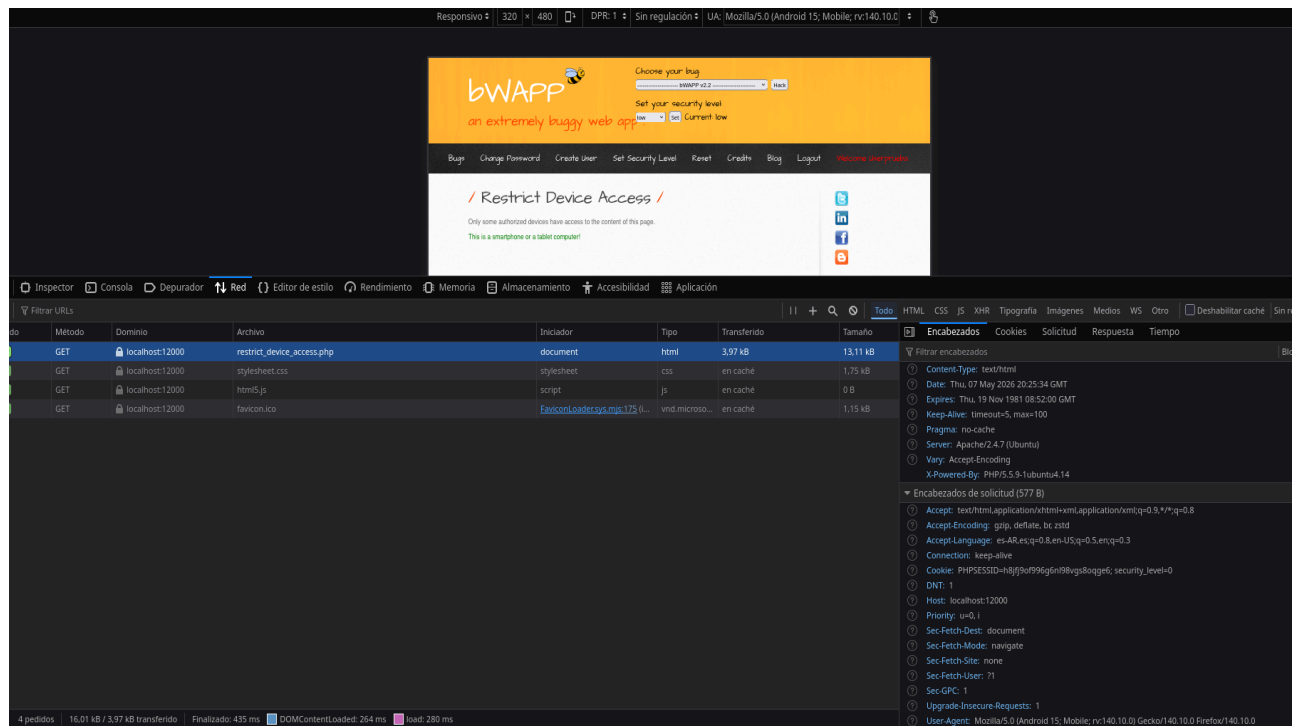


Encabezados de solicitud (570 B)
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: es-AR;q=0.8,en-US;q=0.5,en;q=0.3
Connection: keep-alive
Cookie: PHPSESSID=h8jfj9of996g6nl98vgs8oqqe6; security_level=0
DNT: 1
Host: localhost:12000
Priority: u=0, i
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Sec-GPC: 1
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

El que nos interesa a nosotros es el encabezado User-Agent que especifica el software utilizado por el usuario, es decir, de que medio proviene el requerimiento. En nuestro caso, dado que lo estamos probando desde Firefox, nos aparecerá User-Agent:Mozilla y luego Linux.

Pero si activamos el Responsive Design Mode (ctrl + shift + m) podemos acceder a una interfaz la cuál simula el acceso desde un dispositivo (podemos elegir que dispositivo queremos simular).

Por lo tanto, si ahora refrescamos la página podemos ver no sólo que pudimos acceder al sitio web sin restricciones, sino que ahora User-Agent cambió a “Mozilla/5.0 (Android 15; Mobile; rv:140.10.0) Gecko/140.10.0 Firefox/140.10.0”, es decir, el requerimiento se interpreta como que provino de un dispositivo android versión 15.



Esta es sólo una de las formas de explotar la vulnerabilidad.

¿Cómo podemos solucionarla?

Primero que nada debemos comprender que el User-Agent no funciona como control de seguridad, es decir, no podemos estar seguros restringiendo determinado contenido sólo a dispositivos móviles ya que el agente de usuario puede ser modificado manualmente por el usuario, por lo que NO nos garantiza confiabilidad como método de seguridad.

Si lo que queremos es restringir acceso a contenido sensible, no debemos usar User-Agent, sino autenticación y autorización del lado servidor. El backend debería encargarse de verificar si el usuario tiene permiso o no para visualizar determinado contenido.

Es decir, el frontend o el navegador pueden decir “soy un celular” y el backend NO debería tomar eso como prueba de autorización.

El User-Agent debería tomarse en cuenta sólo para adaptar la interfaz y lograr un diseño responsive, no para restringir acceso a determinado contenido ya que partimos de la premisa de que cualquiera puede simular ser un dispositivo móvil.

Ejercicio 6 - Subir un fix

Cree un branch llamado "fix-practica2"

Modifique el código del ejercicio 1 para que no sea posible obtener los datos personales ni las tarjetas de crédito de otros usuarios. Solamente que se pueda acceder a los datos del usuario logueado.

En el **ejercicio 1** identificamos que el código actual permite acceder a `/perfil/{id}` y ver datos de cualquier usuario cambiando el **ID** en la **URL**.

Eso es el *Broken Access Control*.

Para este ejercicio, primero creamos la rama fix con el comando:

```
git checkout -b fix-practica2
```

En el archivo `app.py` que se encuentra en la carpeta `/ejercicio1` se realizaron los siguientes cambios:

Modificamos el código de la siguiente manera:

```
if usuario_bd["usuario"] == usuario and usuario_bd["contrasena"] == contrasena:
    #return redirect(url_for("perfil", id=usuario_bd["id"]))
    session["user_id"] = usuario_bd["id"] # guardar el ID en sesión
    return redirect(url_for("perfil"))
```

Ahora se guarda el ID del usuario autenticado en la sesión utilizando session de Flask, en lugar de pasarlo por la URL (como quedó comentado en la línea roja del código anterior).

Cambio en la función perfil:

Antes de la modificación el parámetro `id` venía en la **URL**. Eso permitía que cualquier usuario cambiara el número y accediera al perfil de otro (Broken Access Control).

```

@app.route("/perfil/<int:id>", methods=["GET", "POST"])
def perfil(id):
    usuario_bd = None
    for usuario in USUARIOS:
        if usuario["id"] == id:
            usuario_bd = usuario
            break
    if usuario_bd is None:
        return redirect(url_for("login"))

    if request.method == "POST":
        usuario_bd["nombre"] = request.form["nombre"]
        usuario_bd["tarjeta_credito"] = request.form["tarjeta_credito"]
        return redirect(url_for("perfil", id=id))
    return render_template("perfil.html", usuario=usuario_bd)

if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=False)

```

Ahora al eliminar el parámetro `<int:id>` en la ruta y agregar validaciones de sesión para identificar al usuario autenticado, cada usuario solo puede ver y modificar **su propio perfil**, no el de otros.

```

@app.route("/perfil", methods=["GET", "POST"])
def perfil():
    if "user_id" not in session:
        return redirect(url_for("login"))

    id = session["user_id"] # usar el ID guardado en sesión
    usuario_bd = next((u for u in USUARIOS if u["id"] == id), None)

    if usuario_bd is None:
        return redirect(url_for("login"))

    if request.method == "POST":
        usuario_bd["nombre"] = request.form["nombre"]
        usuario_bd["tarjeta_credito"] = request.form["tarjeta_credito"]
        return redirect(url_for("perfil"))

    return render_template("perfil.html", usuario=usuario_bd)

if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=False)

```

Ahora al loguearnos en la app se muestra correctamente la URL sin el id del perfil y solo aparece la información del usuario que se loguea.



Al querer acceder a la información de otro perfil manipulando la URL, ahora ya no se puede.

